

Machine Problem 0

Fall 2025

ECE 391 Course Staff

Due: Fri, 29 August 2025 6:00 PM CDT

Contents

1	Preamble	2
1.1	Suggested reading	2
2	Accessing the work environment	2
2.1	Additional steps (2 & 3 ONLY)	3
2.2	Quick check (<i>optional</i>)	3
2.3	Comments	3
3	Setting up the work-from-home (WFH) environment	3
3.1	On Windows (using WSL 2)	3
3.2	On Linux (most distributions)	5
3.3	On macOS	6
4	Running Star Trek 1971	6
5	GDB	9
6	Handin	10
7	Git: Highly Recommended Knowledge	11
8	Postamble	12

1 Preamble

In this MP, you will get used to the work environment and tools for the class. As mentioned in class, most of your work will be RISC-V code executed on a virtual machine (VM). Several Linux workstations on campus (more on this later) are already configured with all the tools you need for this class. This document will also explain what your work-from-home options are.

The main event of this first machine problem will be getting to run a vintage video game—look forward to it!

1.1 Suggested reading

We encourage you to read a bit about the tools you'll use in the class. You can click on the hyperlinks. These pages can also be found on the [course website](#).

- **QEMU** (`qemu-system-riscv64`) emulates a RISC-V computer (virtualized CPU and peripherals). You will use this very often.
- **GNU C Compiler** (`riscv64-unknown-elf-gcc`) compiles C code for a RISC-V target. You will use this very often.
- **GNU Assembler** (`riscv64-unknown-elf-as`) translates RISC-V assembly to machine code. You will use this sometimes.
- **GNU Make** (`make`) a utility which builds your program (but also does some other things). You will need to modify some `Makefiles` (input to the `make` program) during this class.
- **GNU Debugger** (`riscv-unknown-elf-gdb`) attaches to QEMU, and allows you to debug programs running in the emulator. You will use this very often.
- **Git** (`git`) a [version control](#) tool. You will use this often.

A good understanding of Git, the debugger, and a baseline understanding of each of the components listed above is your main “work” for this MP. Further detail on what material is testable can be found in [Handin](#).

2 Accessing the work environment

There are multiple ways to access a work environment for ECE 391. The officially supported methods are

1. In-person access at ECEB 3026
2. SSH access through `<your_netid_here>@linux.ews.illinois.edu` (not preferred)
3. Remote access through FastX
4. SSH access through `<your_netid_here>@eceb-3026.ews.illinois.edu`

Note that the bottom two methods require using the [Campus VPN](#) or [IllinoisNet](#). If using method 1 or method 4, skip the next section. Otherwise, read the notice which follows.

2.1 Additional steps (2 & 3 ONLY)

Since using FastX or `ssh ...@linux.ews.illinois.edu` gives you a “normal” EWS machine, they will not have the class toolchain enabled automatically. Thus, in each shell you want to use the ECE 391 development tools, you must run:

```
/class/ece391/app/ece391-riscv-dev
```

This will place you into an Apptainer environment containing the class tools. You will need to run this in each shell, or add it to your startup `.bashrc` file.

2.2 Quick check (*optional*)

To check if your toolchain is loaded, you can open a terminal, type `riscv64-` and click **TAB** several times.

You should see a long list of candidate completions for your command appear. If not, refer to the section above or use an official course communication method to request help or information.

2.3 Comments

For all MP demos (when there is), you are (except in extenuating circumstances) required to use the computers in the ECEB 3026 lab.

A work-from-home setup is a helpful development tool, but for consistency and calibration purposes we will grade based on the results yielded on the lab machine. Hence, it would be a good idea to check your work before MP deadlines, even if you do most of your work on your own computer.

3 Setting up the work-from-home (WFH) environment

The work from home environment is not required. You may skip straight to [Running Star Trek 1971](#) if you do not want to setup the work-from-home environment.

3.1 On Windows (using WSL 2)

1. First, follow the instructions [here](#) to install WSL and the Ubuntu distribution of Linux. This setup requires a modern install of WSL 2. If your machine does not support this, please pursue another method for a suitable work environment.
2. Now that you’ve installed WSL 2 with Ubuntu, you should have a new entry in Start called **Start/Ubuntu**. Launching this application should show a terminal with a shell prompt. Verify your installation by running `cat /etc/os-release`. You should see something similar to this:

```
PRETTY_NAME="Ubuntu 24.04 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
```

```
VERSION="24.04 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
```

3. Several packages are necessary. Run this command, which should bring your system up-to-date and install packages we'll need:

```
sudo apt update && sudo apt upgrade -y && \
sudo apt install -y autoconf automake autotools-dev curl \
python3 python3-pip libmpc-dev libmpfr-dev libgmp-dev \
gawk build-essential bison flex texinfo gperf libtool \
patchutils bc zlib1g-dev libexpat-dev ninja-build \
git cmake libglib2.0-dev libslirp-dev libpixman-1-dev libgtk-3-dev
```

4. Now create a directory for your class work, to set up the RISC-V toolchain:

```
mkdir ~/ece391 && cd ~/ece391
git clone --branch 2024.12.16 https://github.com/riscv/riscv-gnu-toolchain
cd riscv-gnu-toolchain
```

5. Now to compile the toolchain. Note that these steps may take a while. Our game plan: Make an `/opt/toolchains/` that we “own”, clone the right version of QEMU and patch it, run `./configure` to enable our favorite features, then run `make`. Before starting, please download `qemu.patch` from the course website, under the **Assignments** tab, and place it in `riscv-gnu-toolchain`.

Ready? Let's go:

```
# cd ~/ece391/riscv-gnu-toolchain, if not already there
# Create toolchain dir, ch(ange) owner
sudo mkdir -p /opt/toolchains/riscv
sudo chown -R $(whoami):$(whoami) /opt/toolchains/riscv
# Clone qemu, apply patch
git clone --depth 1 --branch v9.0.2 https://github.com/qemu/qemu
cd qemu
patch -p0 < ../qemu.patch
# configure and make
./configure --prefix=/opt/toolchains/riscv \
            --target-list=riscv32-softmmu,riscv64-softmmu \
            --enable-gtk --enable-system --disable-werror
make
make install
# go out into toolchain dir, conf+make (can take a very long time)
cd ..
./configure --prefix=/opt/toolchains/riscv/ --enable-multilib
make
```

6. You should now have the toolchain and QEMU, but we still need to add them to **PATH**:

```
echo "PATH=/opt/toolchains/riscv/bin/:$PATH" >> ~/.bashrc
. ~/.bashrc
```

To verify successful installation, run `qemu-system-riscv64`, which should cause a QEMU window to pop up.

Now that you're done, you can move on to [Running Star Trek 1971](#).

3.2 On Linux (most distributions)

This guide assumes you are using a relatively modern version of a well-maintained Linux distribution. We recommend Ubuntu due to its ubiquity, but steps are provided which should work for most Debian-based or RedHat-based distributions too. Note that the process will look quite similar to the WSL setup.

1. Create class directory and clone toolchain:

```
mkdir ~/ece391 && cd ~/ece391
git clone --branch 2024.12.16 https://github.com/riscv/riscv-gnu-toolchain
cd riscv-gnu-toolchain
```

2. Install dependencies. This will vary depending on your distribution. For Ubuntu or Debian Linux, the Step 3 directions from Windows Setup will work. For a RedHat-based distribution, like Fedora, the following should install the necessary dependencies:

```
sudo yum install autoconf automake python3 libmpc-devel mpfr-devel \
    gmp-devel gawk bison flex texinfo patchutils gcc gcc-c++ \
    zlib-devel expat-devel libslirp-devel
```

3. Compiling the toolchain is similar to on WSL. Run the following in your shell:

```
# cd ~/ece391/riscv-gnu-toolchain, if not already there
# Create toolchain dir, ch(ange) owner
sudo mkdir -p /opt/toolchains/riscv
sudo chown -R $(whoami):$(whoami) /opt/toolchains/riscv
# Clone qemu, apply patch
git clone --depth 1 --branch v9.0.2 https://github.com/qemu/qemu
cd qemu
patch -p0 < ../qemu.patch
# configure and make
./configure --prefix=/opt/toolchains/riscv \
    --target-list=riscv32-softmmu,riscv64-softmmu \
    --enable-gtk --enable-system --disable-werror
# the argument after -j defines the number of jobs to run
# this improves build performance on a multicore system
# you can replace $(nproc) with a number of your choice
make -j $(nproc)
make install
```

```
# go out into toolchain dir, conf+make (can take a very long time)
cd ..
./configure --prefix=/opt/toolchains/riscv/ --enable-multilib
make -j $(nproc)
```

4. You should now have the toolchain and QEMU, but we still need to add them to **PATH**:

```
echo "PATH=/opt/toolchains/riscv/bin:$PATH" >> ~/.bashrc
. ~/.bashrc
```

To verify successful installation, run `qemu-system-riscv64`, which should cause a QEMU window to pop up.

Now that you're done, you can move on to [Running Star Trek 1971](#).

3.3 On macOS

Despite the course staff's best attempts, there are certain issues with the macOS setup. Students using macOS should use SSH or FastX.

4 Running Star Trek 1971

Star Trek 1971 is a classic text-based strategy video game. In the game, you control the USS Enterprise and you're on a mission to hunt down and destroy enemy ships.

If you'd like to know more about this hidden gem of a game, check out its [Wikipedia page](#).

To run Star Trek 1971, we need to:

1. Create a repository using the `repo-creator` bot
2. Pull and merge the `main` branch from the "release" repo
3. Run the provided game binary

First, head over to https://edu.cs.illinois.edu/create-gh-repo/fa25_ece391 to create your class repository.

Next, you will need to generate an SSH key to authenticate EWS with GitHub.

1. Generate a public key for your SSH client by running the following on EWS.

```
ssh-keygen -t ed25519
> Enter a file in which to save the key (~/.ssh/id_ed25519):
# press enter
> Enter passphrase (empty for no passphrase):
# type passphrase, or leave empty and press enter
> Enter same passphrase again:
# type the same passphrase again
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

2. Print your public key to the terminal so you can copy it and add it to your GitHub:

```
cat ~/.ssh/id_ed25519.pub
```

3. Now go to your GitHub: <https://github.com/settings/keys>. Select **New SSH Key**.

The screenshot shows the GitHub 'Personal settings' page. On the left is a sidebar with links: Profile, Account, Security, Security log, Emails, Notifications, SSH and GPG keys (highlighted), and Repositories. The main content area has two sections. The 'SSH keys' section has a 'New SSH key' button and text stating 'There are no SSH keys associated with your account.' with links to guides. The 'GPG keys' section has a 'New GPG key' button and text stating 'There are no GPG keys associated with your account.' with a link to a guide.

4. Type in a descriptive title. Paste your copied public key into the key field. Click **Add SSH Key** and type in your GitHub password if prompted.

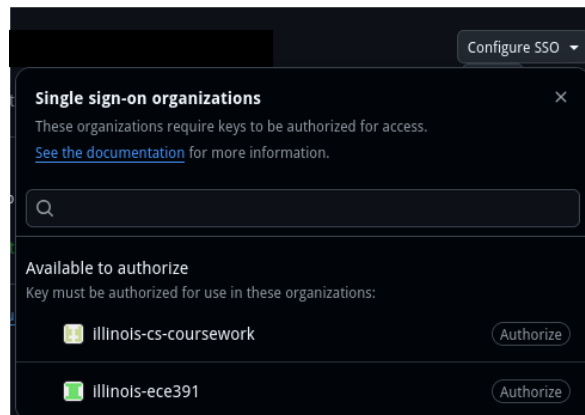
SSH keys / Add new

Title

Key

Add SSH key

5. Authorize the `illinois-ece391` GitHub organization.



Finally, it's time to pull the code:

1. Clone:

```
git clone git@github.com:illinois-ece391/fa25_ece391_YOURNETIDHERE.git
```

2. Change into your new project directory:

```
cd fa25_ece391_YOURNETIDHERE
```

3. Add the `.release` repository as a remote repo:

```
git remote add release git@github.com:illinois-ece391/.release.git
```

4. Now pull the `mp0` materials to your `main` branch

```
git config pull.rebase false
git pull release main --allow-unrelated-histories
```

After this, you should have a folder on your repository called `mp0` containing all the files you need for `mp0`.

5. Launch the game!

```
# enter the mp0 folder first
cd mp0
```

```
# make the executable file
make
```

```
# launch!
qemu-system-riscv64 -m 128M -machine virt -bios none -kernel kernel.elf -nographic
```

```
# or, utilizing the power of Makefiles (equivalent to above)
make run
```

You now have a working copy of *Star Trek 1971*. Be sure to play the game, both for fun and to confirm that the functionality is working.

5 GDB

GDB will be your most important tool to debug your program in ECE391. Although you may feel more comfortable using `print` statements from your past experience with coding, the scope of `print` statements will be limited in ECE391. Therefore, we want to ensure that all students know how to use GDB to debug their code.

GDB is an external program running parallel to your code that will have access to see the current state of your program. You can step through each line of code using GDB, set break points, print out expressions, check register values, and many more usages for you to discover.

In this MP, we will ask you to find some information about a program using GDB.

1. Create a new file `/home/<your_netid_here>/config/gdb/gdbinit` and add the following line.

```
set auto-load safe-path /
```

If you have a WFH setup you might need to put it in some other directory. Proceed to run the GDB, and you will see a prompt on what you will need to do.

2. Launch **TWO** terminals and go the `mp0` directory
3. In one terminal, start running the code. Check the Makefile to see how this is different with `make run`! (Hint: search for `-S` and `-s` flags in the QEMU documentation)

```
make debug-gdb
```

4. In the second terminal, start the GDB tool from RISCv-toolchain with the file `gdb_main.elf` as your executable.

```
riscv64-unknown-elf-gdb gdb_main.elf
```

```
# A GDB console should start
```

5. A GDB console should start now. Attach to the QEMU debugging server you started in the other terminal.

```
(gdb) target remote :1234
```

You should see something like:

```
Remote debugging using :1234
```

For the following steps, you might wish to check the GDB Reference Card on the course website under References to learn about the exact commands. For some steps, you will be asked to reveal some information about the current program state, which you will fill in the blanks in the `submission.txt` file for submission. Because we will ask for the program counter `pc`, you CANNOT modify any code, otherwise you may obtain the incorrect value.

1. To begin, set a breakpoint at the label/function `get_secret` and at line 28 of `gdb_main.c`. Then continue the program.
2. At this point, the program should be at `get_secret`. **What is the current value of the `a2` register in decimal?** (Hint: print the expression `$a2` or use `info reg` command) Then continue the program again.
3. In the QEMU terminal (not the GDB console), enter your NETID in **all lowercase** and hit enter. This should be before you reached the next breakpoint.
4. At this point, the program should be at line 28 of `gdb_main.c`. Step forward by 3 assembly instructions (not lines! instructions are NOT the same as lines). **What is the current value of the `pc` register in hexadecimal?**
5. **What is the instruction that `pc` points to?** (Hint: you can access memory at a specific memory address and format using the `x` command) This should be a `jalr` instruction that composes the `call` to `kprintf`.
6. **What is the instruction that `pc-9` points to?** Notice that this is not a valid instruction because `pc-9` is misaligned!
7. **What is the value of the `secret` variable in hexadecimal?**
8. **Using a `sizeof` expression inside GDB, print out the size of the struct `struct my_struct`.** Notice that the struct is padded (the size is NOT 6 bytes).
9. Using an expression inside GDB, cast the variable `secret` as a `struct my_struct`. **What are the values of the `a` and `b` fields?**

When you are done, you should have a completed `submission.txt` file that looks like the below. These are not the answers.

```
$a2=1234
$pc=0x80001234
jalr -1234(ab)
.insn 1, 0x1234
secret=0x1234567812345678
size=1
a=123456789
b="ab"
```

6 Handin

To get points for this MP, you must complete the following:

- ☐ Have a working RISC-V toolchain and QEMU environment.
- ☐ Follow the GDB practice and fill in the blanks in `submission.txt` in the `mp0` folder of your repository's `main` branch.

- ☐ Create a file called `academic_integrity.txt` in the `mp0` folder of your repository's `main` branch, and type in the text from the image below.
- ☐ Push the above two files to your Github repository by deadline.

For MP0 through MP2, I may not discuss the programming assignments with others, nor look at other students' code (not even a peek in lab).

Starting with MP3, I may only discuss the programming assignments with other members of my group.

For all MPs, I may only consult the permitted online references listed on the course page.

For all MPs, I may not use GPT or other AI-based code assistants.

By submitting this file, I acknowledge that I understand the above policies.

7 Git: Highly Recommended Knowledge

For all ECE 391 assignments, you'll need to know how to use Git. All repositories will be stored in a University of Illinois organization on GitHub. Your individual work will use the repository you have created above, while setting up the game. The final group work will use different repositories, also hosted on GitHub.

Review the documentation (or a tutorial of your choice, or check out the video game [here!](#)) to acquaint yourself with (minimally) the usage of the following Git commands:

```
git init
git clone
git branch
git remote
git commit
git pull
git push
git merge
git reset
git revert
git diff
git stash
```

8 Postamble

Note that under new (as of Spring 2025) policy, EWS Labs will clear your home directory prior to the start of a semester. It's always a good idea to backup any sources you wish to keep in your own personal data storage.

If you would like to understand virtual machine organization, how the environment works or have other questions about the course toolchain, visit office hours or ask a member of course staff!

Finally, welcome to ECE 391 Fall 2025!